

C++ STL

Complete Notes for DSA & Competitive Programming

Standard Template Library — Beginner to Interview Ready

16 Sections • All Containers • Algorithms • Complexity Tables • Revision Sheets

Target: B.Tech CSE (AI & ML) — DSA + Competitive Programming

SECTION 1: ITERATORS

1.1 What is an Iterator?

An iterator is an object that points to an element inside a container. Think of it like a cursor in a text editor — it can point to any character (element), move forward or backward, and let you read or modify what it's pointing at.

★ IMPORTANT

Iterator = Smart Pointer that knows how to move through a container
Every STL container provides iterators
Iterators are the BRIDGE between algorithms and containers

Real-Life Analogy

Imagine a bookshelf with books. An iterator is like your finger — it points to one book at a time. You can move it to the next book, the previous book, or check which book it's pointing to.

Types of Iterators

Type	Can Do	Example Container
Input Iterator	Read only, move forward once	istream
Output Iterator	Write only, move forward once	ostream
Forward Iterator	Read/write, move forward	forward_list
Bidirectional	Read/write, move forward & backward	list, set, map
Random Access	Everything + jump to any position	vector, deque, array

1.2 begin() and end()

begin() returns an iterator to the FIRST element. end() returns an iterator to ONE PAST the last element (not the last element itself!).

NOTE

end() does NOT point to the last element. It points to a phantom position after the last element. Dereferencing end() is UNDEFINED BEHAVIOR.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v = {10, 20, 30, 40, 50};

    auto it = v.begin(); // points to 10
    auto en = v.end();    // points PAST 50

    cout << *it << endl; // Output: 10

    // Iterating with iterators
    for (auto it = v.begin(); it != v.end(); it++) {
        cout << *it << " ";
    }
    // Output: 10 20 30 40 50
    return 0;
}
```

1.3 Dereferencing (*it)

To ACCESS the value at the iterator's position, use the dereference operator *. It's exactly like dereferencing a pointer.

```
vector<int> v = {5, 10, 15};
auto it = v.begin();
```

```
cout << *it;      // 5  - read value
*it = 99;         // modify value
cout << *it;      // 99
```

1.4 Iterator Movement

Operation	Meaning	Works On
<code>it++</code> or <code>++it</code>	Move to next element	All forward iterators
<code>it--</code> or <code>--it</code>	Move to previous element	Bidirectional & Random Access
<code>it + n</code>	Jump n positions forward	Random Access only (vector, deque)
<code>it - n</code>	Jump n positions backward	Random Access only
<code>it1 - it2</code>	Distance between two iterators	Random Access only

```
vector<int> v = {1, 2, 3, 4, 5};
auto it = v.begin();
it++;      // now points to 2
it += 2;   // now points to 4
cout << *it; // 4
```

1.5 Range-Based For Loop

The range-based for loop (C++11) is the cleanest way to iterate. Internally it uses iterators.

```
vector<int> v = {1, 2, 3, 4, 5};

// Read only (copy)
for (auto x : v) {
    cout << x << " ";
}

// Read & modify (reference)
for (auto& x : v) {
    x *= 2; // doubles each element
}
```

1.6 auto vs auto&

Syntax	What Happens	Use Case
<code>auto x : v</code>	Copy of each element (slow for large objects)	Read-only, small types like int
<code>auto& x : v</code>	Reference to actual element (fast)	Modify elements, large objects like strings
<code>const auto& x : v</code>	Const reference (safe, fast read)	Read-only, large objects

 **COMMON MISTAKE**

Forgetting & when you want to modify: `for(auto x : v) { x=0; }` → modifies the COPY, not the actual vector!
Always use `auto&` when modifying elements in range-based for loops

1.7 distance() — Count Steps Between Iterators

`distance(first, last)` returns the number of steps from first to last.

```
#include <iterator>    // for distance()

vector<int> v = {10, 20, 30, 40, 50};
auto it1 = v.begin();
auto it2 = v.begin() + 3;
cout << distance(it1, it2);    // Output: 3
```

Container Type	distance() Complexity
vector, array, deque	O(1) — Random Access
list, set, map	O(n) — must traverse step by step

1.8 advance() — Move Iterator by N Steps

`advance(it, n)` moves the iterator `it` forward by `n` positions (or backward if `n` is negative and iterator supports it).

```
#include <iterator>

list<int> lst = {1, 2, 3, 4, 5};
auto it = lst.begin();    // points to 1
advance(it, 3);           // move forward 3 steps
cout << *it;              // Output: 4

// For random access iterators, just use it += n instead
vector<int> v = {1,2,3,4,5};
auto vit = v.begin();
vit += 3;                 // points to 4
```

QUICK REVISION

- Iterator = smart pointer to element in container
- `begin()` → first element | `end()` → ONE PAST last
- * dereferences, ++ moves forward, -- moves backward (bidirectional+)
- `auto x` copies, `auto&` references (use & to modify)
- `distance(a,b)` = steps from a to b | `advance(it,n)` = move n steps
- Random access iterators support +, -, [] operations

SECTION 2: VECTOR

2.1 What is a Vector?

A vector is a dynamic array — like a regular array but it can GROW and SHRINK automatically. It stores elements in contiguous memory (side by side), just like an array.

★ IMPORTANT

vector = Dynamic Array with automatic resizing
Header: `#include <vector>`
Memory: Contiguous (elements are next to each other in RAM)
Access: $O(1)$ random access by index — `v[0]`, `v[1]`, ...

Real-Life Analogy

Think of a vector like a parking lot. Initially it has say 4 spots. When all spots fill up and a new car comes, the lot DOUBLES its size (moves to a bigger lot), copying all cars there. This is called reallocation.

2.2 Initialization Methods

```
#include <vector>
using namespace std;

vector<int> v1; // empty vector
vector<int> v2(5); // 5 elements, all 0
vector<int> v3(5, 10); // 5 elements, all 10: {10,10,10,10,10}
vector<int> v4 = {1, 2, 3, 4, 5}; // initializer list
vector<int> v5(v4); // copy of v4
vector<int> v6(v4.begin(), v4.begin()+3); // {1,2,3} from v4

// 2D vector
vector<vector<int>> grid(3, vector<int>(4, 0)); // 3x4 grid of zeros
```

2.3 Common Operations & Functions

Function	Description	Return Type	Time
<code>v.push_back(x)</code>	Add x at end	void	Amortized $O(1)$
<code>v.pop_back()</code>	Remove last element	void	$O(1)$
<code>v.size()</code>	Number of elements	size_t	$O(1)$
<code>v.empty()</code>	Is vector empty?	bool	$O(1)$
<code>v.front()</code>	First element	ref	$O(1)$
<code>v.back()</code>	Last element	ref	$O(1)$
<code>v[i]</code> or <code>v.at(i)</code>	Element at index i	ref	$O(1)$
<code>v.insert(it, x)</code>	Insert x before iterator it	iterator	$O(n)$
<code>v.erase(it)</code>	Remove element at iterator it	iterator	$O(n)$
<code>v.clear()</code>	Remove all elements	void	$O(n)$

Function	Description	Return Type	Time
<code>v.resize(n)</code>	Resize to n elements	void	O(n)
<code>v.reserve(n)</code>	Reserve space for n elements	void	O(n)
<code>v.capacity()</code>	Current allocated capacity	size_t	O(1)
<code>v.assign(n, val)</code>	Assign n copies of val	void	O(n)
<code>v.swap(v2)</code>	Swap contents with v2	void	O(1)

2.4 Detailed Explanations with Code

`push_back()` and `pop_back()`

```
vector<int> v;
v.push_back(10);    // v = {10}
v.push_back(20);    // v = {10, 20}
v.push_back(30);    // v = {10, 20, 30}
v.pop_back();       // v = {10, 20} - removes 30

cout << v.back();   // 20 (last element)
cout << v.front();  // 10 (first element)
```

`size()` vs `capacity()`

```
vector<int> v;
v.reserve(10);      // allocate space for 10 but size=0
cout << v.size();    // 0
cout << v.capacity(); // 10

v.push_back(1); v.push_back(2);
cout << v.size();    // 2
cout << v.capacity(); // 10 (no reallocation needed)
```

NOTE

Use `reserve()` when you know approximate size in advance — avoids expensive reallocations and makes code faster!

`insert()` and `erase()`

```
vector<int> v = {1, 2, 3, 4, 5};

// insert at position (before iterator)
v.insert(v.begin() + 2, 99); // v = {1, 2, 99, 3, 4, 5}

// erase single element
v.erase(v.begin() + 3);      // v = {1, 2, 99, 4, 5}

// erase range [begin, end)
v.erase(v.begin(), v.begin() + 2); // v = {99, 4, 5}
```

`at()` vs `[]`

```
vector<int> v = {10, 20, 30};
cout << v[1];           // 20 — NO bounds check, FASTER
cout << v.at(1);         // 20 — bounds check, throws exception if out of range
cout << v.at(5);         // throws std::out_of_range exception
cout << v[5];           // UNDEFINED BEHAVIOR — dangerous!
```

2.5 Iteration Methods

```
vector<int> v = {1, 2, 3, 4, 5};

// Method 1: Index-based
for (int i = 0; i < v.size(); i++)
    cout << v[i] << " ";

// Method 2: Range-based for (preferred)
for (auto x : v)
    cout << x << " ";

// Method 3: Iterator
for (auto it = v.begin(); it != v.end(); it++)
    cout << *it << " ";

// Method 4: Reverse iteration
for (auto it = v.rbegin(); it != v.rend(); it++)
    cout << *it << " "; // 5 4 3 2 1
```

2.6 Internal Working — How Vector Grows

push_back calls	size	capacity (typical)
0	0	0
1	1	1
2	2	2
3	3	4
5	5	8
9	9	16

NOTE

Vector doubles its capacity when it runs out of space. This is called geometric growth and makes `push_back()` $O(1)$ amortized — sometimes slow (on reallocation) but fast on average.

COMMON MISTAKE

`v[i]` does NOT check bounds — use `v.at(i)` to be safe while debugging
 After `erase()`, `insert()`, or `push_back()` that reallocates — all iterators become INVALID
`v.size()` returns `size_t` (unsigned). Writing `v.size()-1` when `v` is empty gives HUGE positive number!
 Never do: `for(int i=0; i<=v.size()-1; i++)` when `v` might be empty — use `i < v.size()`

2.7 Interview / DSA Usage

- Store adjacency list: `vector<vector<int>> adj(n)`
- Sliding window problems
- Prefix sum arrays
- DP tables
- Sorting + binary search
- Most common container in competitive programming

QUICK REVISION

- vector = dynamic array, contiguous memory
- `push_back` $O(1)$ amortized, random access $O(1)$
- insert/erase in middle = $O(n)$ — shifts elements
- `reserve()` to avoid reallocation overhead
- `size()` = current elements, `capacity()` = allocated space
- `at()` has bounds check, `[]` does not — use `[]` in CP, `at()` for safety
- `rbegin()/rend()` for reverse iteration

SECTION 3: LIST

3.1 Linked List Concepts

Singly Linked List

Each node holds data and a pointer to the NEXT node. Can only traverse forward. Insertion/deletion at known position is $O(1)$, but you must walk to find the position first.

Doubly Linked List

Each node holds data, a pointer to NEXT node, AND a pointer to PREVIOUS node. Can traverse both ways. STL `list<T>` is a doubly linked list.

IMPORTANT

STL `list<T>` = Doubly Linked List
Header: `#include <list>`
NO random access (no `[]` operator)
Fast insert/delete at ANY position if you have an iterator: $O(1)$
Slow search: $O(n)$ — must walk from beginning

Real-Life Analogy

Think of a train — each coach is connected to the next (and previous) coach. You can easily add or remove a coach in the middle by just re-connecting the links. But to find coach #7 you have to walk through coaches 1,2,3... one by one.

3.2 Initialization

```
#include <list>
using namespace std;

list<int> l1; // empty
list<int> l2(5, 0); // {0,0,0,0,0}
list<int> l3 = {1, 2, 3, 4, 5}; // initializer list
list<int> l4(l3); // copy
```

3.3 Common Operations

Function	Description	Time
<code>l.push_back(x)</code>	Add x at end	$O(1)$
<code>l.push_front(x)</code>	Add x at beginning	$O(1)$
<code>l.pop_back()</code>	Remove last element	$O(1)$
<code>l.pop_front()</code>	Remove first element	$O(1)$
<code>l.front()</code>	Access first element	$O(1)$
<code>l.back()</code>	Access last element	$O(1)$
<code>l.insert(it, x)</code>	Insert x before iterator it	$O(1)$
<code>l.erase(it)</code>	Remove element at iterator it	$O(1)$
<code>l.remove(val)</code>	Remove ALL occurrences of val	$O(n)$
<code>l.sort()</code>	Sort the list	$O(n \log n)$
<code>l.reverse()</code>	Reverse the list	$O(n)$
<code>l.unique()</code>	Remove consecutive duplicates	$O(n)$
<code>l.merge(l2)</code>	Merge two sorted lists	$O(n)$
<code>l.size()</code>	Number of elements	$O(1)$
<code>l.empty()</code>	Is list empty?	$O(1)$

3.4 Code Examples

```
#include <iostream>
#include <list>
using namespace std;

int main() {
    list<int> l = {3, 1, 4, 1, 5, 9};

    l.push_front(0); // {0, 3, 1, 4, 1, 5, 9}
    l.push_back(2);  // {0, 3, 1, 4, 1, 5, 9, 2}
    l.pop_front();   // {3, 1, 4, 1, 5, 9, 2}
```

```

l.pop_back();      // {3, 1, 4, 1, 5, 9}

// insert at 3rd position
auto it = l.begin();
advance(it, 2);    // move to 3rd element
l.insert(it, 99);  // {3, 1, 99, 4, 1, 5, 9}

l.remove(1);       // remove ALL 1s: {3, 99, 4, 5, 9}

l.sort();          // {3, 4, 5, 9, 99}
l.reverse();       // {99, 9, 5, 4, 3}

for (auto x : l) cout << x << " ";
// Output: 99 9 5 4 3
return 0;
}

```

3.5 List vs Vector Comparison

Feature	vector	list
Memory	Contiguous	Non-contiguous (nodes)
Random access (v[i])	O(1) ✓	Not available ✗
Insert at front	O(n)	O(1) ✓
Insert in middle (with it)	O(n)	O(1) ✓
Search	O(n) or O(log n)	O(n)
Cache friendliness	Excellent ✓	Poor (pointer chasing)
Memory overhead	Low	High (2 pointers per node)
Sort	std::sort() O(n log n)	l.sort() O(n log n)

NOTE

In practice, vector is almost always faster due to cache efficiency. Use list only when you need O(1) insertions/deletions at known positions AND you already have an iterator to that position.

COMMON MISTAKE

list has NO [] operator — l[2] won't compile
std::sort() won't work on list — use l.sort() member function
advance() on list is O(n), not O(1) like vector

QUICK REVISION

- list = doubly linked list, non-contiguous memory
- push_front/push_back/pop_front/pop_back all O(1)
- insert/erase at iterator position = O(1)
- NO random access — no l[i] operator

- Use `l.sort()` not `std::sort()` for lists
- `remove(val)` deletes ALL occurrences of `val`
- Prefer vector unless you specifically need $O(1)$ middle insertions

SECTION 4: DEQUE (Double-Ended Queue)

4.1 What is a Deque?

Deque (pronounced 'deck') stands for Double-Ended Queue. It's like a vector but you can efficiently add/remove from BOTH ends. Internally stored in fixed-size chunks (not fully contiguous like vector).

★ IMPORTANT

deque = Dynamic array with fast front AND back operations
 Header: `#include <deque>`
 Supports `[]` random access like vector
`push_front` and `pop_front` are $O(1)$ — unlike vector where they are $O(n)$

Real-Life Analogy

A deque is like a queue at a movie theater where people can join or leave from BOTH the front AND the back — not just one end.

4.2 Deque vs Vector

Feature	vector	deque
Memory layout	Single contiguous block	Multiple fixed-size chunks
<code>push_back</code>	$O(1)$ amortized	$O(1)$ amortized
<code>push_front</code>	$O(n)$ (shifts all)	$O(1)$ ✓
<code>pop_front</code>	$O(n)$	$O(1)$ ✓
Random access <code>[]</code>	$O(1)$	$O(1)$
Cache efficiency	Better	Slightly worse
<code>reserve()</code>	Supported	Not supported
Memory overhead	Low	Moderate

4.3 Common Operations

```
#include <iostream>
```

```

#include <deque>
using namespace std;

int main() {
    deque<int> dq = {3, 4, 5};

    dq.push_front(2);    // {2, 3, 4, 5}
    dq.push_front(1);    // {1, 2, 3, 4, 5}
    dq.push_back(6);     // {1, 2, 3, 4, 5, 6}
    dq.pop_front();      // {2, 3, 4, 5, 6}
    dq.pop_back();       // {2, 3, 4, 5}

    cout << dq[0];       // 2   (random access OK)
    cout << dq.front();   // 2
    cout << dq.back();    // 5
    cout << dq.size();    // 4

    for (auto x : dq) cout << x << " ";
    // Output: 2 3 4 5
    return 0;
}

```

4.4 When to Use Deque?

- When you need to add/remove from BOTH ends efficiently
- Implementing sliding window maximum problems (monotonic deque)
- BFS with priority on both ends
- When `std::queue` or `std::stack` internals need to be inspected (both use deque as default underlying container)

NOTE

`std::queue` and `std::stack` are implemented on top of deque by default in STL!

QUICK REVISION

- deque = double-ended queue with $O(1)$ front and back operations
- Supports [] random access like vector
- `push_front`/`pop_front` are $O(1)$ — major advantage over vector
- Not fully contiguous memory — slightly worse cache performance
- Use when you need efficient both-end operations

SECTION 5: STACK

5.1 LIFO Concept

Stack follows LIFO — Last In, First Out. The last element pushed is the first one popped. Like a stack of plates: you add to the top, and take from the top.

★ IMPORTANT

Stack = LIFO data structure
Header: `#include <stack>`
Implemented on top of deque by default
No iteration — you can only see the TOP element

5.2 Operations

Function	Description	Time
<code>s.push(x)</code>	Add x to top of stack	O(1)
<code>s.pop()</code>	Remove top element	O(1)
<code>s.top()</code>	Access top element	O(1)
<code>s.size()</code>	Number of elements	O(1)
<code>s.empty()</code>	Is stack empty?	O(1)

⚠ COMMON MISTAKE

`pop()` does NOT return the value — it just removes it. Save with `top()` first!
Calling `top()` or `pop()` on empty stack = UNDEFINED BEHAVIOR
Always check `empty()` before `pop()` or `top()`

5.3 Code Example

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> s;

    s.push(10);
    s.push(20);
    s.push(30);
    // Stack (top to bottom): 30, 20, 10

    cout << s.top();    // 30
    s.pop();            // removes 30
    cout << s.top();    // 20
    cout << s.size();   // 2

    // Safe pop pattern:
    while (!s.empty()) {
        cout << s.top() << " ";
    }
```

```
s.pop();  
}  
// Output: 20 10  
return 0;  
}
```

5.4 Interview/DSA Applications

- Balanced parentheses checking
- Next Greater Element
- Infix to Postfix/Prefix conversion
- Undo/Redo operations
- DFS (iterative implementation)
- Monotonic stack problems
- Stock span problem
- Largest rectangle in histogram

QUICK REVISION

- Stack = LIFO — Last In, First Out
- push() adds to top, pop() removes from top, top() peeks top
- pop() does NOT return value — use top() before pop()
- Check empty() before top()/pop()
- Used in: parentheses, NGE, DFS, expression evaluation

SECTION 6: QUEUE

6.1 FIFO Concept

Queue follows FIFO — First In, First Out. The first element added is the first one removed. Like a line at a bank: the person who arrives first gets served first.

IMPORTANT

Queue = FIFO data structure
Header: #include <queue>
Add at back (push), remove from front (pop)
See front element with front(), back with back()

6.2 Operations

Function	Description	Time
q.push(x)	Add x at back of queue	O(1)
q.pop()	Remove front element	O(1)
q.front()	Access front element	O(1)
q.back()	Access back element	O(1)
q.size()	Number of elements	O(1)
q.empty()	Is queue empty?	O(1)

! COMMON MISTAKE

pop() does NOT return the value — use front() first!
Calling front()/pop() on empty queue = UNDEFINED BEHAVIOR
Always check empty() before front()/back()/pop()

6.3 Code Example

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<int> q;

    q.push(10);    // Queue: [10]
    q.push(20);    // Queue: [10, 20]
    q.push(30);    // Queue: [10, 20, 30]

    cout << q.front(); // 10 (first in)
    cout << q.back();  // 30 (last in)
    cout << q.size();   // 3

    q.pop();        // removes 10
    cout << q.front(); // 20

    while (!q.empty()) {
        cout << q.front() << " ";
        q.pop();
    }
    // Output: 20 30
    return 0;
}
```

6.4 DSA Applications

- BFS (Breadth-First Search) — most important use
- Level order traversal of trees
- Scheduling/task management
- Sliding window using deque

- Producer-consumer problems

QUICK REVISION

- Queue = FIFO — First In, First Out
- push() adds to back, pop() removes from front
- front() = first element, back() = last element
- pop() does NOT return value — use front() before pop()
- MOST IMPORTANT use: BFS traversal

SECTION 7: PRIORITY QUEUE

7.1 What is a Priority Queue?

A priority queue is a queue where each element has a priority. The element with the HIGHEST priority is served first. By default in C++ STL, the MAXIMUM element has highest priority (Max Heap).

IMPORTANT

priority_queue = Heap data structure (Max Heap by default)
Header: #include <queue>
top() always gives the MAX element (by default)
Internal structure: Binary Heap (not sorted, but root is max/min)

Real-Life Analogy

Hospital emergency room — patients don't wait in order of arrival. The patient with the most critical condition (highest priority) gets treated first, regardless of when they arrived.

7.2 Max Heap (Default)

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    priority_queue<int> pq; // Max Heap by default

    pq.push(10);
    pq.push(30);
    pq.push(20);
    pq.push(5);

    cout << pq.top(); // 30 (max element)
    pq.pop();
}
```

```

    cout << pq.top(); // 20 (next max)

    // Prints in decreasing order:
    while (!pq.empty()) {
        cout << pq.top() << " ";
        pq.pop();
    }
    // Output: 20 10 5
    return 0;
}

```

7.3 Min Heap

To get a min heap (smallest element at top), use `greater<int>` as comparator:

```

// Min Heap - smallest element at top
priority_queue<int, vector<int>, greater<int>> minpq;

minpq.push(10);
minpq.push(30);
minpq.push(5);
minpq.push(20);

cout << minpq.top(); // 5 (min element)
// Prints in increasing order: 5 10 20 30

```

NOTE

Memory trick: `greater<int>` → 'greater at bottom' → min at top (Min Heap). Default (`less<int>`) → 'less at bottom' → max at top (Max Heap).

7.4 Operations

Function	Description	Time
<code>pq.push(x)</code>	Insert x into heap	$O(\log n)$
<code>pq.pop()</code>	Remove top element	$O(\log n)$
<code>pq.top()</code>	Access top element	$O(1)$
<code>pq.size()</code>	Number of elements	$O(1)$
<code>pq.empty()</code>	Is PQ empty?	$O(1)$

7.5 Priority Queue with Custom Comparator (Pairs)

```

// Min heap of pairs (sorted by first element)
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;

pq.push({5, 'A'});
pq.push({2, 'B'});
pq.push({8, 'C'});

cout << pq.top().first; // 2 (min first element)
cout << pq.top().second; // 'B'

```

7.6 DSA Applications

- Dijkstra's shortest path algorithm
- Prim's MST algorithm
- Huffman encoding
- K largest/smallest elements
- Merge K sorted arrays/lists
- Task scheduling by priority
- A* search algorithm

! COMMON MISTAKE

pq.pop() does NOT return the value — use pq.top() first!
PQ is not sorted — you only get max/min, not ordered traversal
Can't iterate through PQ without destroying it

⚡ QUICK REVISION

- priority_queue = Max Heap by default (top = maximum)
- For Min Heap: priority_queue<int, vector<int>, greater<int>>
- push() = $O(\log n)$, pop() = $O(\log n)$, top() = $O(1)$
- top() gives max/min, pop() removes it
- Key use: Dijkstra, K-th largest/smallest, greedy algorithms
- Memory trick: greater<int> → min heap | less<int> (default) → max heap

SECTION 8: SET

8.1 What is a Set?

A set stores UNIQUE elements in SORTED order automatically. No duplicates allowed. Implemented as a Red-Black Tree (a balanced BST) internally.

★ IMPORTANT

set = Sorted, Unique elements container
Header: #include <set>
All elements automatically sorted in ascending order
NO duplicates — inserting a duplicate does nothing
All operations: $O(\log n)$ — BST property

Real-Life Analogy

Think of a set like a VIP guest list — each person appears only once, and they are alphabetically ordered. Adding the same person twice has no effect.

8.2 Operations

Function	Description	Time
<code>s.insert(x)</code>	Insert <code>x</code> (ignored if duplicate)	$O(\log n)$
<code>s.erase(x)</code>	Remove element with value <code>x</code>	$O(\log n)$
<code>s.erase(it)</code>	Remove element at iterator <code>it</code>	$O(\log n)$
<code>s.find(x)</code>	Iterator to <code>x</code> , or <code>end()</code> if not found	$O(\log n)$
<code>s.count(x)</code>	1 if <code>x</code> exists, 0 if not	$O(\log n)$
<code>s.lower_bound(x)</code>	Iterator to first elem $\geq x$	$O(\log n)$
<code>s.upper_bound(x)</code>	Iterator to first elem $> x$	$O(\log n)$
<code>s.size()</code>	Number of elements	$O(1)$
<code>s.empty()</code>	Is set empty?	$O(1)$
<code>s.clear()</code>	Remove all elements	$O(n)$
<code>s.begin()/s.end()</code>	Iterators (traverse in sorted order)	$O(1)$

8.3 Code Example

```
#include <iostream>
#include <set>
using namespace std;

int main() {
    set<int> s;

    s.insert(5);
    s.insert(2);
    s.insert(8);
    s.insert(2);    // duplicate – ignored
    s.insert(1);

    // Set is always sorted: {1, 2, 5, 8}
    for (auto x : s) cout << x << " ";
    // Output: 1 2 5 8

    // find()
    auto it = s.find(5);
    if (it != s.end()) cout << "Found: " << *it;    // Found: 5

    // count() – membership check
    cout << s.count(2);    // 1 (exists)
    cout << s.count(7);    // 0 (not found)

    // lower_bound and upper_bound
    auto lb = s.lower_bound(3);    // first  $\geq 3$  → points to 5
    auto ub = s.upper_bound(5);    // first  $> 5$  → points to 8
    cout << *lb;    // 5
```

```

cout << *ub; // 8

// erase
s.erase(2); // removes 2
// Set is now: {1, 5, 8}
return 0;
}

```

NOTE

lower_bound(x) = first element $\geq x$ upper_bound(x) = first element $> x$ To find if x exists: use count(x) or find(x) != end()

8.4 DSA Usage

- Remove duplicates from an array
- Check if element exists in $O(\log n)$
- Find floor/ceil of a number
- Maintaining sorted unique collection
- Dijkstra's algorithm (set of {dist, node})

COMMON MISTAKE

Cannot modify elements in set — erase and re-insert instead
 set<int> sorted ASCENDING by default. For descending: set<int, greater<int>>
 Iterators are bidirectional, not random access — no s.begin()+3

QUICK REVISION

- set = sorted unique elements, implemented as Red-Black BST
- All operations $O(\log n)$: insert, erase, find, count, lower/upper_bound
- count() returns 0 or 1 (since no duplicates)
- lower_bound(x) = first $\geq x$ | upper_bound(x) = first $> x$
- Traversal always gives sorted order
- For descending: set<int, greater<int>>

SECTION 9: MULTISSET

9.1 What is a Multiset?

A multiset is exactly like a set but **ALLOWS DUPLICATE ELEMENTS**. All elements are still stored in sorted order. Internally also a Red-Black Tree, but allows multiple nodes with same key.

★ IMPORTANT

multiset = Sorted container that ALLOWS duplicates

Header: `#include <set>`

`count(x)` returns how many times `x` appears

`erase(x)` removes ALL occurrences of `x` — be careful!

9.2 Operations

Function	Description	Time
<code>ms.insert(x)</code>	Insert <code>x</code> (duplicates allowed)	$O(\log n)$
<code>ms.erase(x)</code>	Remove ALL occurrences of <code>x</code>	$O(\log n + \text{count})$
<code>ms.erase(ms.find(x))</code>	Remove ONE occurrence of <code>x</code>	$O(\log n)$
<code>ms.find(x)</code>	Iterator to first <code>x</code> , or <code>end()</code>	$O(\log n)$
<code>ms.count(x)</code>	Number of times <code>x</code> appears	$O(\log n + \text{count})$
<code>ms.lower_bound(x)</code>	Iterator to first elem $\geq x$	$O(\log n)$
<code>ms.upper_bound(x)</code>	Iterator to first elem $> x$	$O(\log n)$

⚠ COMMON MISTAKE

`ms.erase(val)` removes ALL copies of `val`! To remove just ONE, use: `ms.erase(ms.find(val))`

This is the most common multiset mistake in interviews and contests!

9.3 Code Example

```
#include <iostream>
#include <set> // multiset is in <set>
using namespace std;

int main() {
    multiset<int> ms;

    ms.insert(5);
    ms.insert(2);
    ms.insert(5);
    ms.insert(3);
    ms.insert(5);

    // Sorted with duplicates: {2, 3, 5, 5, 5}
    for (auto x : ms) cout << x << " ";
    // Output: 2 3 5 5 5

    cout << ms.count(5); // 3
    cout << ms.count(10); // 0

    // Remove ONE occurrence of 5
    ms.erase(ms.find(5)); // {2, 3, 5, 5}

    // Remove ALL 5s
    ms.erase(5); // {2, 3}
```

```
for (auto x : ms) cout << x << " ";  
// Output: 2 3  
return 0;  
}
```

9.4 DSA Usage

- Sliding window median
- Maintaining sorted multiset of values for range queries
- Count frequency of elements in sorted manner

QUICK REVISION

- multiset = sorted container allowing duplicates
- count(x) = frequency of x in the multiset
- erase(x) removes ALL x — use erase(find(x)) for ONE removal
- lower/upper_bound work same as set
- All operations $O(\log n)$

SECTION 10: UNORDERED_SET

10.1 What is an Unordered Set?

An unordered_set stores UNIQUE elements with NO particular order. Uses hashing internally, which gives $O(1)$ average time for insert, find, and erase.

IMPORTANT

unordered_set = Hash Set — $O(1)$ avg operations, no order
Header: #include <unordered_set>
Average case $O(1)$, Worst case $O(n)$ — when many hash collisions
Elements NOT sorted — random order when iterating

Hashing Concept (Simple)

A hash function takes a value (like an integer or string) and converts it to an index (bucket number). Element is stored in that bucket. When you search, the same hash function tells you exactly which bucket to look in — $O(1)$!

10.2 set vs unordered_set

Feature	set	unordered_set
Order	Sorted ascending	No order
Underlying	Red-Black BST	Hash Table
Insert	$O(\log n)$	$O(1)$ avg
Find	$O(\log n)$	$O(1)$ avg
Erase	$O(\log n)$	$O(1)$ avg
Worst case	$O(\log n)$	$O(n)$ — hash collision
lower_bound	Available	NOT available
Memory	Less	More (hash table)
Use when	Need sorted order or lb/ub	Just need fast lookup

10.3 Code Example

```
#include <iostream>
#include <unordered_set>
using namespace std;

int main() {
    unordered_set<int> us;

    us.insert(5);
    us.insert(2);
    us.insert(8);
    us.insert(2); // duplicate - ignored

    cout << us.count(2); // 1 (exists)
    cout << us.count(7); // 0 (not found)

    auto it = us.find(5);
    if (it != us.end()) cout << "Found!" << endl;

    us.erase(2);
    cout << us.count(2); // 0

    // Iteration - ORDER NOT GUARANTEED
    for (auto x : us) cout << x << " ";
    // Could output: 8 5 OR 5 8 - undefined order
    return 0;
}
```

NOTE

In competitive programming, `unordered_set` with certain inputs can be hacked to cause $O(n^2)$ behavior. For safety in contests, you can use a custom hash or just use `set`.

COMMON MISTAKE

NO `lower_bound`/`upper_bound` in `unordered_set` — use `set` for that

Iteration order is unpredictable
Worst case $O(n)$ with many hash collisions

QUICK REVISION

- `unordered_set` = hash table, no order, $O(1)$ avg operations
- insert, find, erase, count — all $O(1)$ average
- No `lower_bound/upper_bound` — that's set's job
- Use when you need fast lookup and don't care about order
- Avoid when you need sorted traversal or range queries

SECTION 11: MAP

11.1 What is a Map?

A map stores KEY-VALUE pairs where each key is UNIQUE and sorted. Like a dictionary — each word (key) has exactly one definition (value). Implemented as Red-Black BST on keys.

IMPORTANT

map = Sorted dictionary with unique keys
Header: `#include <map>`
Keys are always sorted in ascending order
Each key maps to exactly ONE value
All operations: $O(\log n)$

Real-Life Analogy

Think of a phone book — each name (key) maps to a phone number (value). Names are sorted alphabetically. No two entries have the same name.

11.2 Initialization and Insertion

```
#include <map>
using namespace std;

map<string, int> m;

// Method 1: [] operator (inserts with default value if not exists)
m["apple"] = 5;
m["banana"] = 3;
m["cherry"] = 8;

// Method 2: insert() with pair
```

```
m.insert({"date", 2});
m.insert(make_pair("elderberry", 7));

// Method 3: initializer list
map<string, int> m2 = {"x", 1}, {"y", 2}, {"z", 3};
```

11.3 Accessing Values

```
map<string, int> m = {"a", 1}, {"b", 2}, {"c", 3};

// [] operator – creates entry with 0 if key doesn't exist!
cout << m["a"];    // 1
cout << m["d"];    // 0 – WARNING: inserts {"d", 0} into map!

// at() – throws exception if key not found, does NOT insert
cout << m.at("b"); // 2
// m.at("z");      // throws std::out_of_range
```

COMMON MISTAKE

`m[key]` INSERTS a default value (0, empty string, etc.) if key doesn't exist!
 Use `m.find(key)` or `m.count(key)` to check existence before accessing
 Use `m.at(key)` when you want an exception instead of silent insertion

11.4 Common Operations

Function	Description	Time
<code>m[key] = val</code>	Insert/update key-value pair	$O(\log n)$
<code>m.insert({k,v})</code>	Insert if key not present	$O(\log n)$
<code>m.at(key)</code>	Access value (throws if missing)	$O(\log n)$
<code>m.find(key)</code>	Iterator to key, or <code>end()</code>	$O(\log n)$
<code>m.count(key)</code>	1 if key exists, 0 if not	$O(\log n)$
<code>m.erase(key)</code>	Remove entry by key	$O(\log n)$
<code>m.erase(it)</code>	Remove entry by iterator	$O(\log n)$
<code>m.lower_bound(k)</code>	First entry with key $\geq k$	$O(\log n)$
<code>m.upper_bound(k)</code>	First entry with key $> k$	$O(\log n)$
<code>m.size()</code>	Number of key-value pairs	$O(1)$
<code>m.empty()</code>	Is map empty?	$O(1)$
<code>m.clear()</code>	Remove all entries	$O(n)$

11.5 Traversal

```
map<string, int> m = {"c", 3}, {"a", 1}, {"b", 2};

// Range-based for – always sorted by key
```

```

for (auto& [key, val] : m) { // C++17 structured binding
    cout << key << ":" << val << " ";
}
// Output: a:1 b:2 c:3 (sorted alphabetically)

// Old style
for (auto it = m.begin(); it != m.end(); it++) {
    cout << it->first << " " << it->second;
}

```

11.6 Frequency Counting (Most Common DSA Pattern!)

```

string s = "aabbccccc";
map<char, int> freq;

for (char c : s)
    freq[c]++; // [] creates entry with 0 if missing, then increments

for (auto& [ch, cnt] : freq)
    cout << ch << ": " << cnt << "\n";
// Output:
// a: 2
// b: 3
// c: 4

```

✓ REMEMBER

The `freq[x]++` pattern is one of the most used patterns in competitive programming. The `[]` operator initializes to 0 if missing, then increments — perfect for counting!

⚡ QUICK REVISION

- `map` = sorted key-value pairs, unique keys, Red-Black BST
- `m[key]` inserts default value if key missing — use carefully
- `find(key) != end()` to check existence safely
- Traversal always in sorted key order
- `freq[x]++` is the classic frequency counting pattern
- All operations $O(\log n)$
- Structured binding: `for(auto& [k,v] : m)` — C++17

SECTION 12: MULTIMAP

12.1 What is a Multimap?

A multimap is like a map but **ALLOWS DUPLICATE KEYS**. One key can map to multiple values. All entries still sorted by key.

★ IMPORTANT

multimap = Sorted map allowing duplicate keys
Header: `#include <map>`
NO [] operator — use `insert()` always
`find()` returns iterator to ONE of the matching entries (not all)
`equal_range(key)` gives range of all entries with that key

12.2 Code Example

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    multimap<string, int> mm;

    mm.insert({"fruit", 1});
    mm.insert({"fruit", 2});
    mm.insert({"fruit", 3});
    mm.insert({"veggie", 10});

    cout << mm.count("fruit");    // 3

    // Get all values for a key using equal_range
    auto range = mm.equal_range("fruit");
    for (auto it = range.first; it != range.second; it++)
        cout << it->second << " ";
    // Output: 1 2 3

    // Erase all entries with key "fruit"
    mm.erase("fruit");
    cout << mm.count("fruit");    // 0
    return 0;
}
```

Function	Description	Time
<code>mm.insert({k,v})</code>	Insert key-value pair	$O(\log n)$
<code>mm.find(key)</code>	Iterator to first occurrence of key	$O(\log n)$
<code>mm.count(key)</code>	Number of entries with this key	$O(\log n + \text{count})$
<code>mm.erase(key)</code>	Remove ALL entries with this key	$O(\log n + \text{count})$
<code>mm.equal_range(key)</code>	Pair of iterators: [begin, end) of key	$O(\log n)$
<code>mm.lower_bound(key)</code>	First entry with key $\geq$ this	$O(\log n)$
<code>mm.upper_bound(key)</code>	First entry with key $>$ this	$O(\log n)$

⚠ COMMON MISTAKE

NO [] operator in multimap — it's ambiguous (which value for a key?)
Always use `insert()` to add entries

`erase(key)` removes ALL entries with that key

⚡ QUICK REVISION

- `multimap` = sorted map with duplicate keys allowed
- No `[]` operator — use `insert({key, val})` always
- `equal_range(key)` gives all entries for a key
- `count(key)` = number of entries with that key
- `erase(key)` removes ALL matching entries

SECTION 13: UNORDERED_MAP

13.1 What is an Unordered Map?

An `unordered_map` stores key-value pairs with NO particular order. Uses hashing on keys for $O(1)$ average operations. Fastest lookup among map containers.

★ IMPORTANT

`unordered_map` = Hash Map — $O(1)$ avg, no order
Header: `#include <unordered_map>`
Most common map in competitive programming for speed
Same `[]` operator behavior as `map` — inserts default if missing

13.2 map vs unordered_map

Feature	map	unordered_map
Order	Sorted by key	No order
Underlying	Red-Black BST	Hash Table
Insert	$O(\log n)$	$O(1)$ avg
Find	$O(\log n)$	$O(1)$ avg
Erase	$O(\log n)$	$O(1)$ avg
Worst case	$O(\log n)$	$O(n)$ — hash collision
<code>lower_bound</code>	Available	NOT available
Custom key types	Need operator<	Need hash function
Use when	Need sorted/range	Just need fast lookup

13.3 Code Example

```
#include <iostream>
#include <unordered_map>
using namespace std;

int main() {
    unordered_map<string, int> um;

    // Insert
    um["apple"] = 5;
    um["banana"] = 3;
    um.insert({"cherry", 8});

    // Access
    cout << um["apple"];    // 5
    cout << um["missing"];  // 0 - INSERTS {"missing", 0}!

    // Safe check
    if (um.count("banana"))
        cout << "Found banana: " << um["banana"];

    // Frequency counting (same pattern as map)
    string s = "hello world";
    unordered_map<char, int> freq;
    for (char c : s) freq[c]++;

    for (auto& [ch, cnt] : freq)
        cout << ch << ":" << cnt << " ";
    // Output order NOT guaranteed
    return 0;
}
```

NOTE

For competitive programming, prefer `unordered_map` over `map` when you don't need sorted order — it's faster! But be careful with worst-case $O(n)$ when inputs are designed to cause collisions.

COMMON MISTAKE

`um[key]` INSERTS default value if key missing — same as `map`
No `lower_bound/upper_bound` in `unordered_map`
String keys work fine; custom struct keys need custom hash function

QUICK REVISION

- `unordered_map` = hash map, $O(1)$ avg, no sorted order
- Same `[]` syntax as `map` — same gotcha: inserts default if missing
- `freq[x]++` works exactly like `map` for counting
- No `lower_bound/upper_bound` — that's `map`'s territory
- Fastest lookup: use when sorted order not needed

SECTION 14: LOWER_BOUND & UPPER_BOUND

14.1 What Are They?

`lower_bound` and `upper_bound` are binary search functions that work on SORTED containers to find positions of elements efficiently in $O(\log n)$.

★ IMPORTANT

`lower_bound(begin, end, x)` → iterator to first element $\geq x$
`upper_bound(begin, end, x)` → iterator to first element $> x$
Both require the range to be SORTED
Return `end()` if no such element exists

Real-Life Analogy

Think of a sorted list of exam scores: 60 65 70 70 70 75 80. If you want to find score 70: `lower_bound` points to the FIRST 70, `upper_bound` points to the position AFTER THE LAST 70 (i.e., 75).

14.2 Syntax

```
#include <algorithm> // for std::lower_bound, std::upper_bound

// For vectors/arrays (use std::)
auto lb = lower_bound(v.begin(), v.end(), x);
auto ub = upper_bound(v.begin(), v.end(), x);

// For set/map (use member function – faster)
auto lb = s.lower_bound(x);
auto ub = s.upper_bound(x);
```

14.3 Comprehensive Example

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {10, 20, 20, 30, 40, 50};
    // index:      0  1  2  3  4  5

    auto lb = lower_bound(v.begin(), v.end(), 20);
    auto ub = upper_bound(v.begin(), v.end(), 20);

    cout << *lb;                // 20 (first >= 20)
    cout << *ub;                // 30 (first > 20)
    cout << lb - v.begin();      // 1  (index of first 20)
    cout << ub - v.begin();      // 3  (index after last 20)
    cout << ub - lb;            // 2  (count of 20s)
```

```

// Check if element exists
auto it = lower_bound(v.begin(), v.end(), 25);
if (it == v.end() || *it != 25)
    cout << "25 not found";           // 25 not found

// Find floor (largest element <= x)
auto floor_it = lower_bound(v.begin(), v.end(), 25);
if (floor_it != v.begin()) {
    floor_it--;
    cout << *floor_it; // 20 (largest element <= 25)
}
return 0;
}

```

14.4 Lower vs Upper Bound Summary

Scenario	lower_bound(x)	upper_bound(x)
x exists	Points to first x	Points to element after last x
x not in array	First element > x	Same — first element > x
x larger than all	end()	end()
x smaller than all	begin()	begin()

14.5 Common Interview Patterns

```

vector<int> v = {10, 20, 30, 40, 50}; // sorted

// Count occurrences of 30
int cnt = upper_bound(v.begin(), v.end(), 30) - lower_bound(v.begin(), v.end(), 30);

// Floor of x (largest element <= x)
auto it = upper_bound(v.begin(), v.end(), 25); // first > 25
if (it != v.begin()) { --it; cout << *it; } // 20

// Ceiling of x (smallest element >= x)
auto it2 = lower_bound(v.begin(), v.end(), 25); // first >= 25
if (it2 != v.end()) cout << *it2; // 25

```

NOTE

For set and map: prefer member functions `s.lower_bound(x)` over `std::lower_bound(s.begin(), s.end(), x)` — the member version is $O(\log n)$, `std::` version on set is $O(n)$!

QUICK REVISION

- `lower_bound` = first element $\geq x$
- `upper_bound` = first element $> x$
- Both return `end()` if not found — always check!
- Count of x: `upper_bound(x) - lower_bound(x)`
- Floor of x: `--upper_bound(x)` | Ceiling of x: `lower_bound(x)`
- Array needs to be SORTED

- Use member functions on set/map — they're $O(\log n)$ not $O(n)$

SECTION 15: STL ALGORITHMS

STL algorithms are general-purpose functions in `<algorithm>` that work on any container through iterators. They separate the algorithm from the data structure.

★ IMPORTANT

Header: `#include <algorithm>`

All algorithms work through iterators

Range is `[begin, end)` — begin inclusive, end exclusive

Most work on sorted data for binary search variants

15.1 `sort()`

```
// Syntax
sort(begin, end);           // ascending order
sort(begin, end, comparator); // custom order

// Examples
vector<int> v = {5, 2, 8, 1, 9, 3};
sort(v.begin(), v.end());    // {1, 2, 3, 5, 8, 9}

// Descending
sort(v.begin(), v.end(), greater<int>()); // {9, 8, 5, 3, 2, 1}

// Custom comparator (sort by absolute value)
sort(v.begin(), v.end(), [](int a, int b) {
    return abs(a) < abs(b);
});

// Sort array (not vector)
int arr[] = {5, 3, 1, 4, 2};
sort(arr, arr + 5);

// Sort strings
vector<string> words = {"banana", "apple", "cherry"};
sort(words.begin(), words.end()); // alphabetical
```

sort() detail	Value
Time complexity	$O(n \log n)$ — Introsort (hybrid QuickSort + HeapSort + InsertionSort)
Space complexity	$O(\log n)$
Stable?	NO — equal elements may be reordered
<code>stable_sort()</code>	$O(n \log^2 n)$ — preserves relative order of equal

sort() detail	Value
	elements

15.2 reverse()

```
// Syntax
reverse(begin, end);

vector<int> v = {1, 2, 3, 4, 5};
reverse(v.begin(), v.end()); // {5, 4, 3, 2, 1}

// Reverse part of vector
reverse(v.begin(), v.begin() + 3); // reverse first 3 elements

// Reverse a string
string s = "hello";
reverse(s.begin(), s.end()); // s = "olleh"
```

Complexity	Value
Time	$O(n)$
Space	$O(1)$

15.3 find()

```
// Syntax
auto it = find(begin, end, value);
// Returns iterator to first occurrence, or end() if not found

vector<int> v = {10, 20, 30, 40, 50};
auto it = find(v.begin(), v.end(), 30);

if (it != v.end())
    cout << "Found at index: " << it - v.begin(); // 2
else
    cout << "Not found";
```

Complexity	Value
Time	$O(n)$ — linear search, container need NOT be sorted
Space	$O(1)$

COMMON MISTAKE

`std::find()` is $O(n)$ LINEAR search — not binary search. For sorted containers, use `binary_search()` or `lower_bound()` for $O(\log n)$.

15.4 count()

```
// Syntax
int cnt = count(begin, end, value);

vector<int> v = {1, 2, 2, 3, 2, 4, 2};
cout << count(v.begin(), v.end(), 2); // 4

string s = "hello world";
cout << count(s.begin(), s.end(), 'l'); // 3
```

Complexity	Value
Time	O(n)
Space	O(1)

15.5 max_element() and min_element()

```
// Syntax
auto it = max_element(begin, end);
auto it = min_element(begin, end);
// Returns ITERATOR to max/min, not the value itself

vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6};

auto mx = max_element(v.begin(), v.end());
cout << *mx; // 9 (value)
cout << mx - v.begin(); // 5 (index)

auto mn = min_element(v.begin(), v.end());
cout << *mn; // 1

// max of entire vector (simpler)
cout << *max_element(v.begin(), v.end());
```

Complexity	Value
Time	O(n)
Space	O(1)

15.6 next_permutation()

Rearranges elements to the next lexicographically greater permutation. Returns true if next permutation exists, false if it was the last permutation (and resets to first).

```
#include <algorithm>

// Print all permutations of {1,2,3}
vector<int> v = {1, 2, 3};
sort(v.begin(), v.end()); // MUST be sorted to get all permutations

do {
    for (int x : v) cout << x << " ";
    cout << endl;
} while (next_permutation(v.begin(), v.end()));
```

```
// Output:
// 1 2 3
// 1 3 2
// 2 1 3
// 2 3 1
// 3 1 2
// 3 2 1
```

Complexity	Value
Single call	$O(n)$
All permutations	$O(n! \times n)$

NOTE

Always sort before using `next_permutation` in a do-while loop to get ALL permutations. If not sorted, you'll miss some.

15.7 `binary_search()`

Returns true if element exists in SORTED range, false otherwise. Uses binary search — $O(\log n)$.

```
// Syntax
bool found = binary_search(begin, end, value);

vector<int> v = {1, 2, 3, 4, 5, 6}; // MUST be sorted

cout << binary_search(v.begin(), v.end(), 3); // 1 (true)
cout << binary_search(v.begin(), v.end(), 7); // 0 (false)
```

Complexity	Value
Time	$O(\log n)$ — binary search
Space	$O(1)$

NOTE

`binary_search()` only tells you IF element exists (true/false). To find WHERE, use `lower_bound()` instead.

15.8 Other Useful Algorithms

Algorithm	Purpose	Complexity
<code>accumulate(b, e, init)</code>	Sum of elements (init=0 for sum)	$O(n)$
<code>fill(b, e, val)</code>	Fill range with value	$O(n)$
<code>copy(b, e, dest)</code>	Copy range to destination	$O(n)$
<code>unique(b, e)</code>	Remove consecutive duplicates (sort first)	$O(n)$

Algorithm	Purpose	Complexity
<code>rotate(b, mid, e)</code>	Rotate so mid becomes first	$O(n)$
<code>nth_element(b, nth, e)</code>	Partition around nth element	$O(n)$ avg
<code>partial_sort(b, mid, e)</code>	Sort first (mid-b) elements	$O(n \log k)$
<code>any_of(b, e, pred)</code>	True if any element satisfies pred	$O(n)$
<code>all_of(b, e, pred)</code>	True if all elements satisfy pred	$O(n)$
<code>none_of(b, e, pred)</code>	True if no elements satisfy pred	$O(n)$

```
#include <numeric> // for accumulate

vector<int> v = {1, 2, 3, 4, 5};

// Sum
int sum = accumulate(v.begin(), v.end(), 0); // 15

// any_of / all_of
bool anyEven = any_of(v.begin(), v.end(), [](int x){ return x%2==0; }); // true
bool allPos  = all_of(v.begin(), v.end(), [](int x){ return x>0; });    // true

// unique (removes consecutive duplicates)
vector<int> u = {1,1,2,2,3,3,3,4};
sort(u.begin(), u.end());
auto newEnd = unique(u.begin(), u.end());
u.erase(newEnd, u.end()); // u = {1, 2, 3, 4}
```

QUICK REVISION

- `sort()` = $O(n \log n)$ Introsort — use `greater<>` for descending
- `reverse()` = $O(n)$ | `find()` = $O(n)$ linear | `count()` = $O(n)$
- `max_element/min_element` return ITERATORS — dereference with `*`
- `binary_search()` = $O(\log n)$ true/false — needs sorted range
- `next_permutation` — sort first, use in do-while for all perms
- `lower_bound` faster than `find` for sorted ranges
- `accumulate()` in `<numeric>` for sums

SECTION 16: MASTER REVISION SHEET

Use this section for last-minute revision before exams, contests, or interviews.

16.1 Container Quick Reference

Container	Header	Order?	Duplicates?	Access	Key Operations
vector	<vector>	Insertion	Yes	O(1) []	push_back, pop_back, size
list	<list>	Insertion	Yes	No []	push_front/back, pop_front/back
deque	<deque>	Insertion	Yes	O(1) []	push/pop front & back
stack	<stack>	LIFO	Yes	top() only	push, pop, top
queue	<queue>	FIFO	Yes	front/back	push, pop, front
priority_queue	<queue>	Heap	Yes	top() only	push, pop, top
set	<set>	Sorted	No	No []	insert, erase, find, lb, ub
multiset	<set>	Sorted	Yes	No []	insert, erase(find(x)), count
unordered_set	<unordered_set>	None	No	No []	insert, erase, find, count
map	<map>	Sorted key	No	O(log n) []	insert, find, erase, []
multimap	<map>	Sorted key	Yes	No []	insert, equal_range, erase
unordered_map	<unordered_map>	None	No	O(1) avg []	insert, find, erase, []

16.2 Time Complexity Cheat Sheet

Operation	vector	list	deque	set/map	unordered_set/ map	priority_queue
Access by index	O(1)	O(n)	O(1)	N/A	N/A	N/A
Insert at end	O(1)*	O(1)	O(1)*	O(log n)	O(1) avg	O(log n)
Insert at front	O(n)	O(1)	O(1)	N/A	N/A	N/A
Insert in middle	O(n)	O(1)†	O(n)	O(log n)	O(1) avg	N/A
Delete at end	O(1)	O(1)	O(1)	O(log n)	O(1) avg	O(log n)
Delete at front	O(n)	O(1)	O(1)	N/A	N/A	N/A
Search	O(n)	O(n)	O(n)	O(log n)	O(1) avg	N/A
Sort	O(n log n)	O(n log n)	O(n log n)	Always sorted	Not sorted	Always ordered

* = amortized | † = O(1) only if you already have the iterator

16.3 Iterator Cheat Sheet

Operation	Syntax	Note
Begin	<code>v.begin()</code>	Iterator to first element
End	<code>v.end()</code>	One PAST last — never dereference
Reverse begin	<code>v.rbegin()</code>	Iterator to last element
Reverse end	<code>v.rend()</code>	One before first
Dereference	<code>*it</code>	Get value at iterator
Advance	<code>it++</code> or <code>++it</code>	Move to next element
Go back	<code>it--</code> or <code>--it</code>	Bidirectional+ only
Jump forward	<code>it += n</code> or <code>it + n</code>	Random access only (vector, deque)
Distance	<code>distance(a, b)</code>	Steps from a to b
Move n steps	<code>advance(it, n)</code>	Works on any forward iterator
To index	<code>it - v.begin()</code>	Random access only

16.4 Algorithm Cheat Sheet

Algorithm	Syntax	Time	Needs Sorted?	Returns
<code>sort</code>	<code>sort(b,e) / sort(b,e,comp)</code>	$O(n \log n)$	No (sorts it)	void
<code>reverse</code>	<code>reverse(b,e)</code>	$O(n)$	No	void
<code>find</code>	<code>find(b,e,val)</code>	$O(n)$	No	iterator
<code>count</code>	<code>count(b,e,val)</code>	$O(n)$	No	count
<code>binary_search</code>	<code>binary_search(b,e,val)</code>	$O(\log n)$	YES	bool
<code>lower_bound</code>	<code>lower_bound(b,e,val)</code>	$O(\log n)$	YES	iterator
<code>upper_bound</code>	<code>upper_bound(b,e,val)</code>	$O(\log n)$	YES	iterator
<code>max_element</code>	<code>max_element(b,e)</code>	$O(n)$	No	iterator
<code>min_element</code>	<code>min_element(b,e)</code>	$O(n)$	No	iterator
<code>next_permutation</code>	<code>next_permutation(b,e)</code>	$O(n)$	Sort first	bool
<code>accumulate</code>	<code>accumulate(b,e,init)</code>	$O(n)$	No	value
<code>unique</code>	<code>unique(b,e)</code>	$O(n)$	Sort first	new end
<code>fill</code>	<code>fill(b,e,val)</code>	$O(n)$	No	void

16.5 Frequently Used Syntax Patterns

```
// — FREQUENCY COUNT —————
map<int,int> freq;
for (int x : arr) freq[x]++;

// — REMOVE DUPLICATES FROM VECTOR —————
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());

// — K-TH LARGEST ELEMENT —————
priority_queue<int> pq(v.begin(), v.end()); // max heap
for(int i=1; i<k; i++) pq.pop();
int kth = pq.top();

// — K-TH SMALLEST (MIN HEAP) —————
priority_queue<int,vector<int>,greater<int>> minpq(v.begin(),v.end());
for(int i=1; i<k; i++) minpq.pop();
int kth = minpq.top();

// — COUNT ELEMENTS IN RANGE [l, r] in sorted vector
int cnt = upper_bound(v.begin(),v.end(),r) - lower_bound(v.begin(),v.end(),l);

// — FLOOR AND CEILING —————
// Floor: largest element <= x
auto it = upper_bound(v.begin(), v.end(), x);
if (it != v.begin()) { --it; cout << *it; }

// Ceiling: smallest element >= x
auto it2 = lower_bound(v.begin(), v.end(), x);
if (it2 != v.end()) cout << *it2;

// — GRAPH ADJACENCY LIST —————
int n = 5;
vector<vector<int>> adj(n);
adj[0].push_back(1); adj[1].push_back(0); // undirected edge 0-1

// — BFS TEMPLATE —————
queue<int> q;
vector<bool> visited(n, false);
q.push(start); visited[start] = true;
while (!q.empty()) {
    int node = q.front(); q.pop();
    for (int nb : adj[node]) {
        if (!visited[nb]) { visited[nb]=true; q.push(nb); }
    }
}

// — BALANCED PARENTHESES (STACK) —————
stack<char> st;
bool balanced = true;
for (char c : s) {
    if (c=='(' || c=='[' || c=='{') st.push(c);
    else {
        if (st.empty()) { balanced=false; break; }
        char top = st.top(); st.pop();
        if ((c==')' && top!='(') || (c==']' && top!='[') || (c=='}' && top!='{'))
            { balanced=false; break; }
    }
}
if (!st.empty()) balanced = false;
```

16.6 STL Interview Tips

- Always state the time complexity of your approach
- Know when to use set vs unordered_set: need sorting/range → set, need speed → unordered_set
- Know that map[] inserts default value — mention this in interviews
- For Dijkstra: use priority_queue with Min Heap {dist, node}
- For BFS: always use queue. For DFS: use stack (iterative) or recursion
- vector is cache-friendly — prefer over list unless you truly need O(1) middle insertions
- pop() never returns value — always top()/front() before pop()
- lower_bound on set — use member function s.lower_bound(x), NOT std::lower_bound on set
- multiset: erase(find(x)) for ONE removal; erase(x) removes ALL
- next_permutation: sort first; use in do-while for all permutations

16.7 STL for DSA — Which Container to Use?

Problem Type	Use This	Why
Store sorted unique elements	set	O(log n) insert/search, always sorted
Count word frequencies	unordered_map	O(1) avg, freq[word]++ pattern
Find k-th largest	priority_queue	Max heap top() = largest
Find k-th smallest	priority_queue (min heap)	Min heap top() = smallest
BFS traversal	queue	FIFO property
DFS traversal	stack	LIFO property
Balanced parentheses	stack	Match opening/closing
Next Greater Element	stack	Monotonic stack
Range sum queries	vector + prefix	O(1) query after O(n) build
Sliding window max/min	deque	Monotonic deque O(1)
Graph adjacency list	vector<vector<int>>	Dynamic sizing per node
Store sorted key-value pairs	map	Sorted, unique keys
Fast key-value lookup	unordered_map	O(1) avg, no ordering needed
Maintain multiset of elements	multiset	Sorted with duplicates
Binary search on array	lower_bound/ upper_bound	O(log n)

16.8 Common Pitfalls Summary

COMMON MISTAKE

vector: `v[i]` has no bounds check, `v.at(i)` does
Any `pop()`: does NOT return value — save with `top()/front()` first
map/unordered_map: `m[key]` inserts default if missing
multiset: `erase(val)` removes ALL, use `erase(find(val))` for ONE
set iterators: NOT random access — can't do `s.begin()+3`
lower/upper_bound: requires SORTED range
`end()` iterator: NEVER dereference — it's past the last element
Range-based for without `&`: modifies copy, not actual element
list: use `l.sort()` not `std::sort()` — `std::sort` needs random access
priority_queue: by default MAX heap — use `greater<>` for MIN heap
next_permutation: SORT first to get all permutations
`distance()` on list/set: $O(n)$ not $O(1)$ — use sparingly

QUICK REVISION

- VECTOR: Dynamic array, $O(1)$ back ops, $O(n)$ front/middle
- LIST: Doubly linked, $O(1)$ everywhere with iterator, no `[]`
- DEQUE: Double-ended, $O(1)$ front+back, supports `[]`
- STACK: LIFO — push/pop/top all $O(1)$
- QUEUE: FIFO — push/pop/front/back all $O(1)$
- PRIORITY QUEUE: Heap — $O(\log n)$ push/pop, $O(1)$ top
- SET: Sorted+unique, $O(\log n)$ all ops, bidirectional iterator
- MULTISSET: Sorted+duplicates, `erase(find(x))` for one removal
- UNORDERED_SET: Hash set, $O(1)$ avg, no order, no lb/ub
- MAP: Sorted KV, unique keys, `m[]` inserts default if missing
- MULTIMAP: Sorted KV, dup keys, use `equal_range`
- UNORDERED_MAP: Hash map, $O(1)$ avg, freq counting king
- LOWER_BOUND: first $\geq x$ | UPPER_BOUND: first $> x$
- SORT: $O(n \log n)$ | FIND: $O(n)$ | BINARY_SEARCH: $O(\log n)$